Goal: Derive backpropagation

First step: Define and analyze functions

Definitions:

L is the number of layers

l is the index of a layer in the network and is written in a superscript

n^l is the number of neurons in layer l

i refers to the ith neuron in a layer and is written in a subscript

$w^l_{j,i}$ is the numerical weight of the connection between neuron j in layer l-1 and neuron i in layer l

a^l_i is the numerical value, called the activation, of neuron i in layer l

z^l_i is the weighted input of neuron i in layer l before applying the sigmoid activation function

b^l_i is the bias (shift) used when calculating the weighted input (z) for neuron i in layer l

The sigmoid function is continuous and differentiable and maps weighted inputs to an activation in the range (0, 1)

The cost function (C) uses half SSE (quadratic cost) to generally represent how the neural network is performing

SSE means sum of squared errors, multiplied by a factor of 1/2 to simplify the derivative

T is the dataset size, t is the index of a piece of data in the dataset

y^t_i is the desired activation of neuron i in the output layer for training example t

$$z^l_i = \sum_{j=1}^{n^{l-1}} (w^l_{j,i} \cdot a^{l-1}_j) + b^l_i$$

$$a^l_i = \sigma(z^l_i), \quad \frac{\partial a^l_i}{\partial z^l_i} = \sigma'(z^l_i)$$

$$\sigma(x) = \frac{1}{1+e^{-x}} = (1+e^{-x})^{-1}$$

$$\sigma'(x) = -1(1+e^{-x})^{-2} \cdot (-e^{-x}) = \frac{e^{-x}}{(1+e^{-x})^2} = \sigma(x)^2 \cdot (\sigma(x)^{-1} - 1) = \sigma(x) \cdot (1 - \sigma(x))$$

$$\frac{\partial a^l_i}{\partial z^l_i} = \sigma'(z^l_i) = a^l_i \cdot (1 - a^l_i)$$

$$C_t = \frac{1}{2} \sum_{i=1}^{n^L} (a^L_i - y^t_i)^2$$

$$C = \frac{\sum_{t=1}^{T} C_t}{T}, \quad \frac{\partial C}{\partial w^l_{j,i}} = \frac{\sum_{t=1}^{T} \frac{\partial C_t}{\partial w^l_{j,i}}}{T}$$

Backpropagation: efficiently compute partial derivatives of cost function with respect to each parameter

Dynamic Programming $\rightarrow$ Reuse already computed subproblems

The partial derivatives of the cost function can be computed by averaging the partial derivatives of each training example

A weight may affect C_t through every computational path from that weight to each reachable neuron in the output layer

To find the cumulative effect on C_t, we must add up the effects of each path

$$\frac{\partial C_t}{\partial a_i^L} = a_i^L - y_i^t$$

$$\frac{\partial C_t}{\partial z_i^L} = (a_i^L - y_i^t) \cdot \frac{\partial a_i^L}{\partial z_i^L} = (a_i^L - y_i^t) \cdot (a_i^L) \cdot (1 - a_i^L)$$

The partial derivative of the cost function with respect to z has a common structure

Each time the chain rule passes through a sigmoid function it contributes the factor $\frac{\partial a_i^l}{\partial z_i^l} = (a_i^l) \cdot (1 - a_i^l)$

We can define and store this output layer partial derivative as the output layer "error"

This is the base case in our dynamic programming problem

$$\delta_i^l := \frac{\partial C_t}{\partial z_i^l}$$

$$\frac{\partial C_t}{\partial z_i^L} = \delta_i^L = (a_i^L - y_i^t) \cdot (a_i^L) \cdot (1 - a_i^L)$$

$$\frac{\partial C_t}{\partial w_{j,i}^L} = \frac{\partial C_t}{\partial z_i^L} \cdot \frac{\partial z_i^L}{\partial w_{j,i}^L} = \delta_i^L \cdot a_j^{L-1}$$

$$\frac{\partial C_t}{\partial b_i^L} = \frac{\partial C_t}{\partial z_i^L} \cdot \frac{\partial z_i^L}{\partial b_i^L} = \delta_i^L \cdot 1 = \delta_i^L$$

Now we can move back one layer and attempt to use the already calculated values

$$\frac{\partial C_t}{\partial w_{j,i}^{L-1}} = \sum_{k=1}^{n^L} \frac{\partial C_t}{\partial z_k^L} \cdot \frac{\partial z_k^L}{\partial a_i^{L-1}} \cdot \frac{\partial a_i^{L-1}}{\partial z_i^{L-1}} \cdot \frac{\partial z_i^{L-1}}{\partial w_{j,i}^{L-1}} = \sum_{k=1}^{n^L} \delta_k^L \cdot w_{i,k}^L \cdot (a_i^{L-1}) \cdot (1 - a_i^{L-1}) \cdot a_j^{L-2}$$

A pattern seems to be emerging, and we may be able to find a general rule for any layer

$$\frac{\partial C_t}{\partial w_{j,i}^{L-1}} = (a_i^{L-1}) \cdot (1 - a_i^{L-1}) \cdot (\sum_{k=1}^{n^L} \delta_k^L \cdot w_{i,k}^L) \cdot a_j^{L-2}$$

$$\delta_i^{L-1} = (a_i^{L-1}) \cdot (1 - a_i^{L-1}) \cdot (\sum_{k=1}^{n^L} \delta_k^L \cdot w_{i,k}^L)$$

$$\frac{\partial C_t}{\partial w_{j,i}^{L-1}} = \delta_i^{L-1} \cdot a_j^{L-2}$$

All the terms that involve k now exist because neuron i in layer l can affect the cost through all k nodes in layer l+1

Now we can iteratively step back and use the information from previous layers to efficiently calculate the next

This is similar to tabulation dynamic programming in which you fill a table iteratively

We already have the base case for the output layer (l=L), so we just need to derive a new formula for all hidden layers

Now for arbitrary l($1 \leq l < L$) :

$$\frac{\partial C_t}{\partial w_{j,i}^l} = \frac{\partial C_t}{\partial z_i^l} \cdot \frac{\partial z_i^l}{\partial w_{j,i}^l} = \delta_i^l \cdot a_j^{l-1} = (a_i^l) \cdot (1 - a_i^l) \cdot (\sum_{k=1}^{n^{l+1}} \delta_k^{l+1} \cdot w_{i,k}^{l+1}) \cdot a_j^{l-1}$$

$$\frac{\partial C_t}{\partial b_i^l} = \frac{\partial C_t}{\partial z_i^l} \cdot \frac{\partial z_i^l}{\partial b_i^l} = \delta_i^l \cdot 1 = \delta_i^l$$

Backpropagation has a time complexity for one training example of O(P) where P is the number of weights and biases

It is only necessary to visit each parameter a constant number of times for each training example

By storing and reusing the similar subproblems represented by δ_i^l, we avoid an exponential time complexity

Without this algorithm larger scale machine learning would likely take too long